PPE Detection and Hazard Monitoring System Using YOLOv8

---

1. Introduction

Workplace safety is a critical requirement in industrial environments such as construction sites, manufacturing plants, and oil and gas facilities. Ensuring the proper use of Personal Protective Equipment (PPE) is essential to minimize risks and prevent accidents.

Manual monitoring of PPE compliance is inefficient, inconsistent, and prone to human error. To address this problem, this project presents a **real-time computer vision-based system** that automatically detects PPE usage, identifies violations, and provides alerts using deep learning techniques.

---

2. Objective

The objective of this assessment was to develop a **real-time, end-to-end computer vision system** that can:

- Detect workers in live video streams
- Identify PPE usage (helmet, glasses, overall)
- Detect hazards (fire and smoke)
- Perform person-wise compliance analysis
- Trigger alerts for violations
- Maintain logs and visual evidence
- Provide a user-friendly interface

---

3. Author Contribution and Implementation Responsibility

This project was completed as part of a **technical assessment**, and the entire system was independently designed and implemented by me.

My responsibilities included:

- Dataset preparation and annotation using Roboflow
- Defining class structure and labeling strategy
- Training and fine-tuning the YOLOv8 model
- Designing person-wise PPE compliance logic
- Implementing real-time detection using OpenCV
- Integrating object tracking (ByteTrack)
- Developing violation detection logic (persistence + cooldown)

- Implementing alert and logging mechanisms
- Designing and building the Streamlit-based user interface

This demonstrates my ability to build a **complete end-to-end computer vision system** from data preparation to deployment.

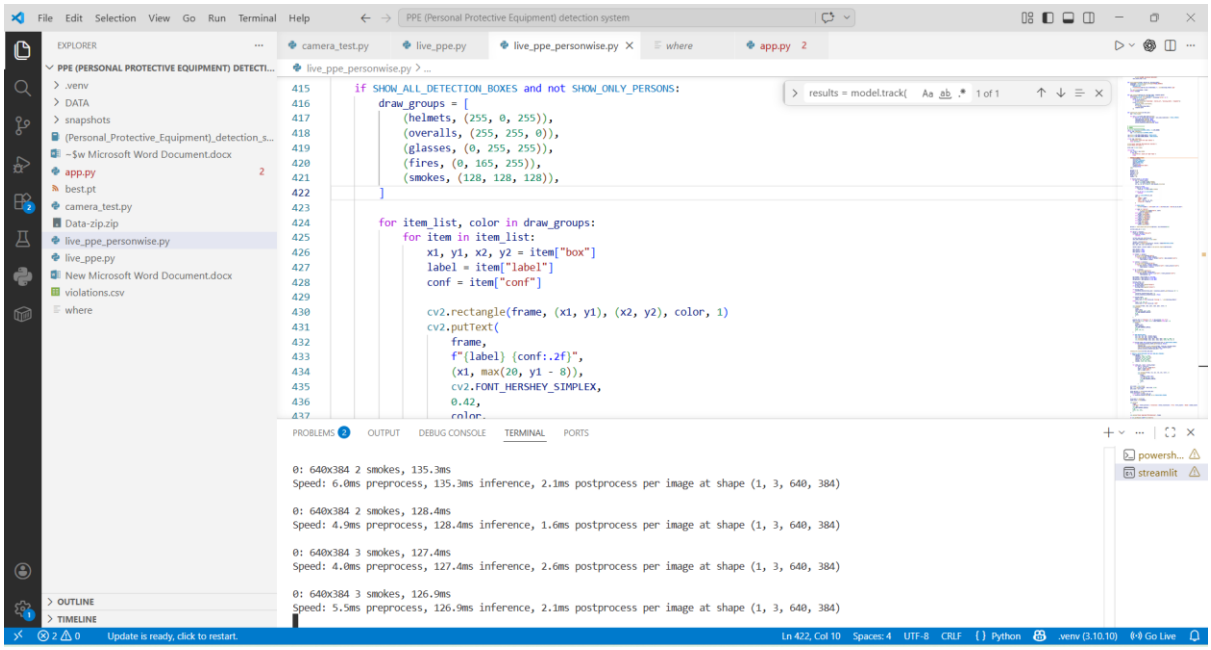
---

4. System Overview

The system follows a complete pipeline:

Input (Camera/Image/Video) → YOLOv8 Model → Detection → Tracking → PPE Analysis → Violation Detection → Alert + Logging → UI Display

This architecture ensures real-time processing and practical usability.



---

5. Dataset Description and Constraints

The model was trained on a dataset of approximately **284 images**.

Dataset Composition:

- PPE-related images (helmet, glasses, overall)
- Hazard scenarios (fire and smoke)
- Background and irrelevant images

Practical Constraint:

Due to the nature of the assessment and time limitations, the dataset size was relatively small and not perfectly curated.

Design Approach:

Instead of restricting the dataset to ideal conditions, I intentionally:

- Included noisy and irrelevant samples
- Retained real-world variability
- Avoided over-cleaning the dataset

Rationale:

This approach was chosen to:

- Simulate real-world deployment conditions
- Improve model robustness
- Reduce overfitting

Impact:

Despite dataset limitations, the system is able to:

- Generalize across different environments
- Perform real-time detection
- Handle imperfect inputs

This demonstrates that a **functional and deployable system can be built under constrained data conditions**.

---

6. Classes Implemented

The system detects the following classes:

- Person
- Helmet
- Glasses
- Overall
- Fire
- Smoke

---

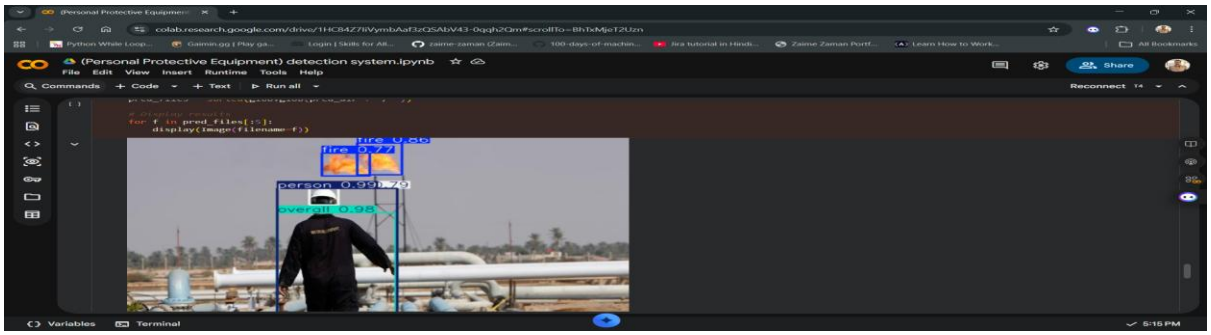
7. Model Training

- Model used: **YOLOv8**

- Framework: Ultralytics
- Training performed on custom dataset
- Best model exported as: **best.pt**

best model download

```
from google.colab import files
files.download("/content/runs/ppe_hazard_model/weights/best.pt")
```



---

8. System Implementation

8.1 Real-Time Detection

- Implemented using OpenCV
- YOLO model processes frames in real time

8.2 Person-wise PPE Analysis

- Each detected person is analyzed individually
- Body regions (head, torso) are used to associate PPE
- Missing PPE is detected per person

8.3 Object Tracking

- ByteTrack algorithm used
- Maintains consistent IDs across frames
- Improves detection stability

8.4 Violation Detection Logic

- Detects missing PPE
- Uses:
    - Frame persistence
    - Cooldown mechanism

8.5 Alert System

- Audible alert triggered when violation is confirmed

8.6 Logging System

- Violations saved in CSV file
- Snapshots saved for evidence

---

9. User Interface

A web-based user interface was developed using Streamlit.

The interface allows users to interact with the system directly through a browser instead of using command-line tools.

Features:

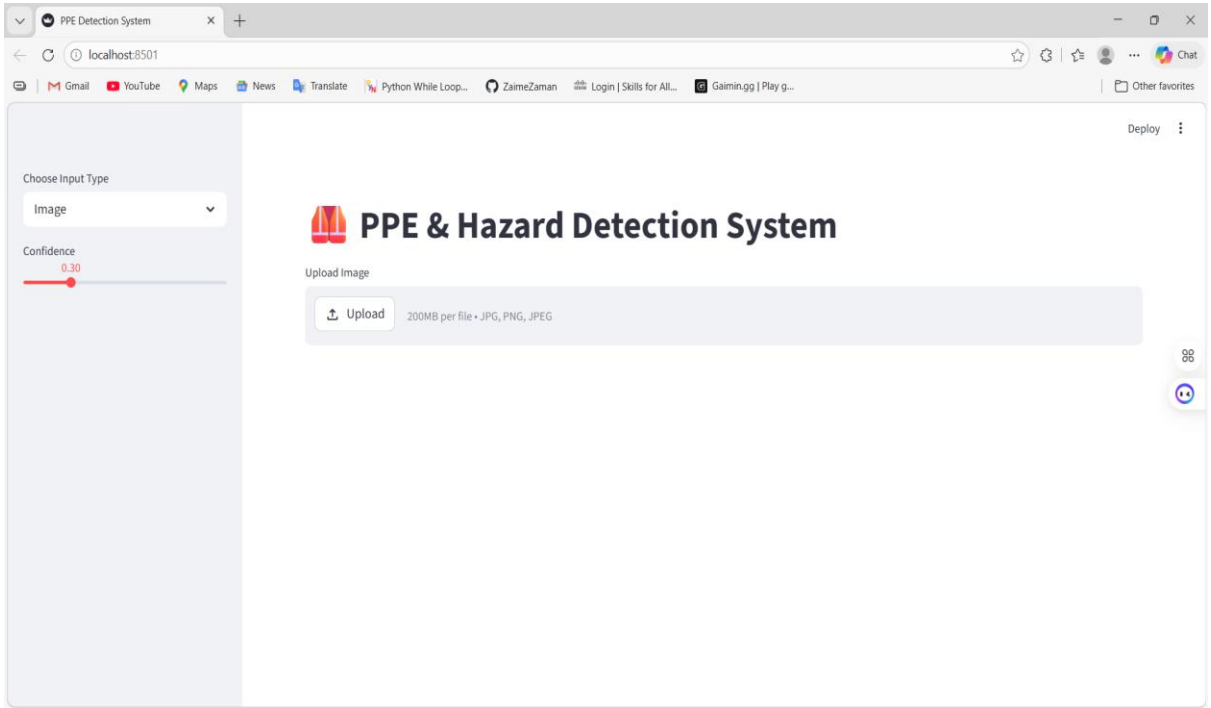
- Image-based detection
- Video-based detection
- Live camera detection
- Adjustable confidence threshold
- Real-time visualization

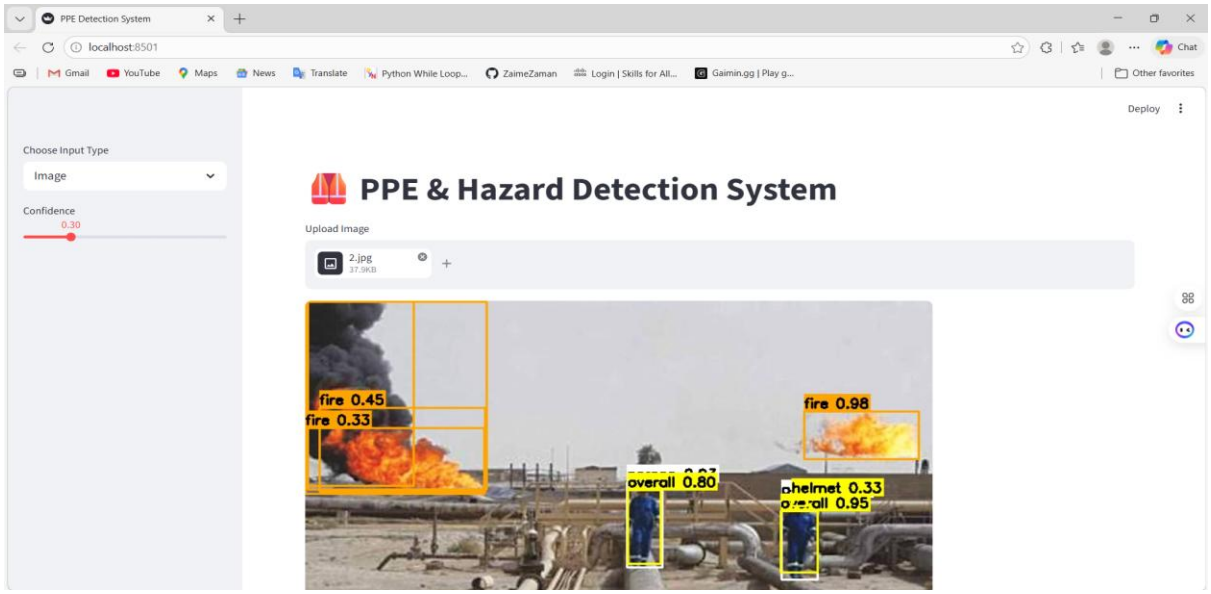
System Flow:

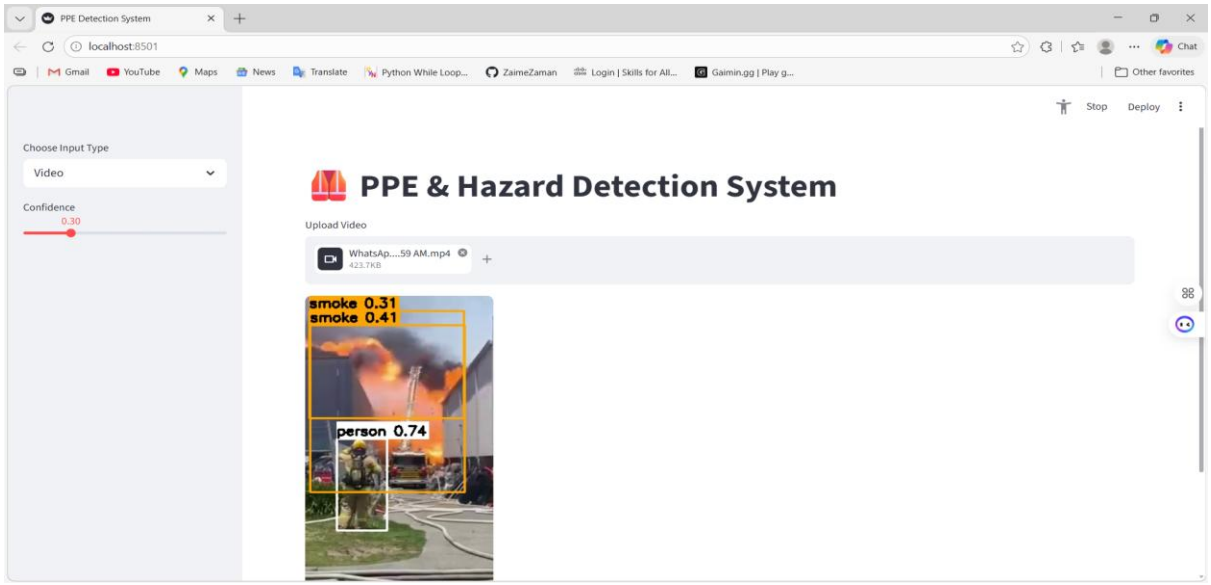
User Input → UI → YOLO Model → Detection → Display Output

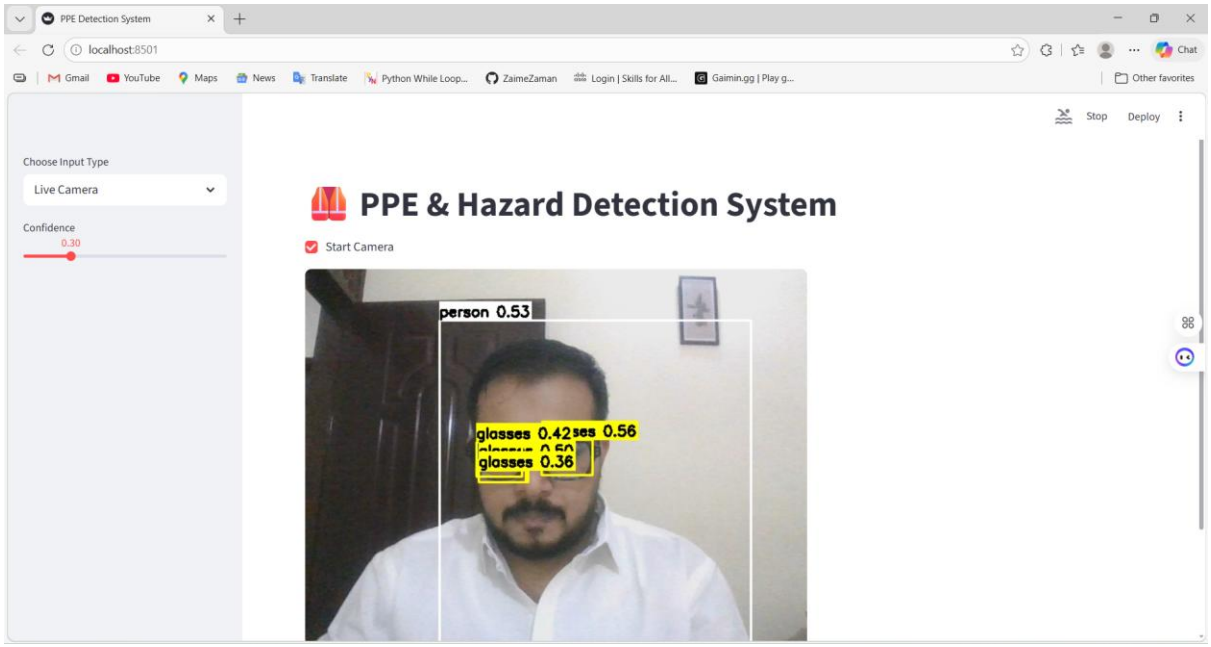
Key Advantage:

The addition of a UI transforms the system into a **complete and deployable application** accessible to non-technical users.




PPE Detection System
localhost:8501
Deploy
Choose Input Type
Image
Confidence
0.30
PPE & Hazard Detection System
Upload Image
2.jpg
fire 0.45
fire 0.33
fire 0.98
overall 0.80
helmet 0.33
overall 0.95


PPE Detection System
localhost:8501
Stop
Deploy
Choose Input Type
Video
Confidence
0.30
PPE & Hazard Detection System
Upload Video
WhatsAp....59 AM.mp4
smoke 0.31
smoke 0.41
person 0.74


PPE Detection System
localhost:8501
Stop
Deploy
Choose Input Type
Live Camera
Confidence
0.30
PPE & Hazard Detection System
Start Camera
person 0.53
glasses 0.42
glasses 0.56
glasses 0.36

---

10. Results

The system successfully performs:

- Multi-person detection
- PPE compliance monitoring
- Hazard detection (fire and smoke)
- Real-time alert generation
- Logging with visual evidence

---

11. Key Achievements

- Developed a complete end-to-end system
- Implemented real-time detection
- Designed person-wise compliance logic
- Integrated tracking and alert mechanisms
- Built a functional UI application

---

12. Limitations and Constraints

- Dataset size (~284 images) is relatively small
- Some classes (e.g., smoke, glasses) have fewer samples
- Gloves and safety boots are not included
- Performance can improve with more data

---

13. Conclusion

This project successfully demonstrates a **real-time PPE detection and hazard monitoring system** using YOLOv8.

The system includes:

- Detection
- Tracking
- Person-wise analysis
- Violation logic
- Alerts

- Logging
- User interface

Despite working under dataset and time constraints, a **complete and functional solution** was developed.

---

14. Project Structure

The project is organized as follows:

- best.pt (trained model)
- app.py (UI application)
- live_ppe.py (real-time detection)
- live_ppe_personwise.py (advanced logic)
- violations.csv (log file)
- snapshots/ (saved violation images)
- dataset files

---

15. Final Statement

This project demonstrates my ability to independently design and implement a real-world computer vision system under practical constraints, covering the full pipeline from data preparation to deployment.